

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_a6bedd49-683\agent-af6d82e3cdfb9a3d4.jsonl`
- SHA-256: `4a8fb6fb21b7fb5a5af632398bfe63ca5fcc8cbe31f7c9e12e87fa89996fec33`
- Source modified: `2026-07-06T23:56:40+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_a6bedd49-683`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T23:48:46.054Z)

Reviewing GitHub PR #55 "F6: persist policy decision traces" (branch feat/f6-policy-decisions -> main). ADR 0012 F6.

+387/-11, 7 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr55 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f6-policy-decisions:<path>').

Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr55/src

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F6 DOES:

- db.py: new policy_decisions table in BOTH _SCHEMA and @_migration(5)
_migration_005_policy_decisions (both CREATE TABLE IF NOT EXISTS).
Schema: (id PK, content_item_id FK content_items, policy_id FK policies, passed INT, trace_json TEXT, evaluated_at TEXT, UNIQUE(content_item_id, policy_id)).
Repos: record_policy_decision (UPSERT on (content_item_id,policy_id):
passed/trace_json/evaluated_at), get_policy_decision, list_policy_decisions_for_content.
_policy_trace_json = json.dumps(trace, sort_keys, compact). _row_to_policy_decision json.loads + validates list.
- models.py: PolicyDecision (frozen). policy.py: stage_results_to_trace(results) -> [{stage_name,passed,reason,metadata}].
- telethon_client.py: _evaluate_policy_stages now returns (passed, results, policy_id).
_record_policy_decision(inserted, policy_id, policy_passed, policy_results):
guards 'if not policy_results or policy_id is None or inserted.id is None: return'; else
db.record_policy_decision(content_item_id=inserted.id, ...).
Called in on_new_message AND backfill AFTER insert_video (so the FK content_item exists).
on_new_message/backfill are '# pragma: no cover'.

VERIFIED by main reviewer: gate green (551 passed @ 94.34%); migration idempotent + reaches user_version=5 on a legacy v1 DB; a full pipeline trace serializes + persists; UPSERT is idempotent (1 row on re-record); policy decisions are stored SEPARATELY from delivery_attempts (addresses an earlier Track C finding).
Note deviations/limits already spotted: schema is one-row-per-decision-with-JSON-trace (ADR literally proposed per-stage rows); NO TTL/cleanup (ADR open question); metadata-rejected (Tier-1 FilterEngine) items get NO policy_decision row (only the evaluate_pipeline Tier 2-4 trace is recorded).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.

Severity: high (migration data loss/corruption or a write bug that crashes ingestion/persists wrong data), medium (real latent bug/FK/serialization crash), low (note/deviation/growth).

DIMENSION: persistence write-path, serialization, and wiring correctness.

Investigate:

- SERIALIZATION crash: record_policy_decision does json.dumps(stage_results_to_trace(results)). All F1-F5 stage metadata is JSON-safe today, but a future

ContentFetcher (F5) could return a StageResult with non-JSON metadata -> json.dumps raises TypeError -> propagates uncaught out of _record_policy_decision -> on_new_message (no try/except) -> crashes ingestion (same class as F5's fetcher-error lesson). Reproduce with a non-serializable metadata value. Severity?

- FK on REJECTED items: a policy-rejected item is still insert_video'd (matched=policy_passed) BEFORE _record_policy_decision, so content_item_id exists.

Confirm a rejected item persists a decision with passed=False and creates NO delivery_attempt. Any path where inserted.id is None but we still try to record?

- Guard 'if not policy_results...': metadata-rejected items (FilterEngine matched=False) skip policy stages -> policy_results=[] -> NO decision row. Confirm and assess for F7 /why (can't explain metadata rejections). Also: matched but no default policy (policy_id None) -> skip (nothing to record) - correct?
- UPSERT/idempotency and evaluated_at (tz-aware ISO, not naive CURRENT_TIMESTAMP - confirm).

trace_json ordering (sort_keys) stable? _row_to_policy_decision

- raises if trace_json isn't a list - can that ever happen from our own writes? Growth/TTL (unbounded - note).

Run repros. Report findings.

2. assistant (2026-07-06T23:48:48.459Z)

I'll review PR #55's persistence write-path, serialization, and wiring. Let me start by reading the relevant files.

3. user (2026-07-06T23:48:50.767Z)

42b5637 feat(policy): persist policy decision traces
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
6d3d7a6 feat(policy): add F2 topic preferences (#51)
---DIFF---
src/tg_video_filter/backfill.py | 13 +++-
src/tg_video_filter/db.py | 120 +++++
src/tg_video_filter/models.py | 16 +++++
src/tg_video_filter/policy.py | 14 ++++
src/tg_video_filter/telethon_client.py | 41 ++++++--
tests/test_db.py | 127 +++++
tests/test_policy.py | 67 ++++++--
7 files changed, 387 insertions(+), 11 deletions(-)

4. assistant (2026-07-06T23:48:52.627Z)

Let me read the core files in the diff.

5. user (2026-07-06T23:48:54.709Z)

```
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index e6fca83..c76747a 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -27,6 +27,7 @@ from tg_video_filter.models import (
     Destination,
     FilterConfig,
     Policy,
+    PolicyDecision,
     RoutingRule,
     Source,
     TelegramSession,
@@ -181,6 +182,16 @@ CREATE TABLE IF NOT EXISTS delivery_attempts (
     next_retry_at TEXT,
     UNIQUE (content_item_id, destination_id)
 );
+
+CREATE TABLE IF NOT EXISTS policy_decisions (
```

```
+ id INTEGER PRIMARY KEY AUTOINCREMENT,
+ content_item_id INTEGER NOT NULL REFERENCES content_items(id),
+ policy_id INTEGER NOT NULL REFERENCES policies(id),
+ passed INTEGER NOT NULL,
+ trace_json TEXT NOT NULL,
+ evaluated_at TEXT NOT NULL,
+ UNIQUE (content_item_id, policy_id)
+);
"""

# Note: the `idx_videos_user_document` partial unique index is created by
@@ -764,6 +775,24 @@ async def _migration_004_destinations_and_routing(conn:
aiosqlite.Connection) ->
    )

+@_migration(5)
+async def _migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
+    """Create ADR 0012 policy decision audit table."""
+    await conn.execute(
+        """
+        CREATE TABLE IF NOT EXISTS policy_decisions (
+            id INTEGER PRIMARY KEY AUTOINCREMENT,
+            content_item_id INTEGER NOT NULL REFERENCES content_items(id),
+            policy_id INTEGER NOT NULL REFERENCES policies(id),
+            passed INTEGER NOT NULL,
+            trace_json TEXT NOT NULL,
+            evaluated_at TEXT NOT NULL,
+            UNIQUE (content_item_id, policy_id)
+        )
+        """
+    )
+
+    async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
+        """Return True if `table` exists (async, PRAGMA table_list / sqlite_master)."""
+        async with conn.execute(
@@ -893,6 +922,20 @@ def _row_to_policy(row: aiosqlite.Row) -> Policy:
    )

+def _row_to_policy_decision(row: aiosqlite.Row) -> PolicyDecision:
+    trace = json.loads(row["trace_json"])
+    if not isinstance(trace, list):
+        raise ValueError("policy_decisions.trace_json must decode to a JSON array")
+    return PolicyDecision(
+        id=row["id"],
+        content_item_id=row["content_item_id"],
+        policy_id=row["policy_id"],
+        passed=bool(row["passed"]),
+        trace=trace,
+        evaluated_at=row["evaluated_at"],
+    )
+
+def _row_to_destination(row: aiosqlite.Row) -> Destination:
+    return Destination(
+        id=row["id"],
@@ -1342,6 +1385,80 @@ async def save_policy_config(
    )

+# --- PolicyDecision repos (ADR 0012, Track F) -----
+
+
+
```

```

+def _policy_trace_json(trace: list[dict[str, object]]) -> str:
+    """Encode a policy stage trace deterministically."""
+    return json.dumps(trace, separators=(",", ":"), sort_keys=True)
+
+
+async def record_policy_decision(
+    conn: aiosqlite.Connection,
+    *,
+    content_item_id: int,
+    policy_id: int,
+    passed: bool,
+    trace: list[dict[str, object]],
+    evaluated_at: datetime | None = None,
+) -> PolicyDecision:
+    """Persist or replace the policy evaluation trace for a content item."""
+    evaluated_at = evaluated_at or _now()
+    trace_json = _policy_trace_json(trace)
+    await conn.execute(
+        """INSERT INTO policy_decisions
+            (content_item_id, policy_id, passed, trace_json, evaluated_at)
+        VALUES (?, ?, ?, ?, ?)
+        ON CONFLICT(content_item_id, policy_id) DO UPDATE SET
+            passed=excluded.passed,
+            trace_json=excluded.trace_json,
+            evaluated_at=excluded.evaluated_at""",
+        (
+            content_item_id,
+            policy_id,
+            int(passed),
+            trace_json,
+            evaluated_at.isoformat(),
+        ),
+    )
+    await conn.commit()
+    decision = await get_policy_decision(conn, content_item_id, policy_id)
+    assert decision is not None
+    return decision
+
+
+async def get_policy_decision(
+    conn: aiosqlite.Connection,
+    content_item_id: int,
+    policy_id: int,
+) -> PolicyDecision | None:
+    """Read a policy decision by its natural key."""
+    conn.row_factory = aiosqlite.Row
+    async with conn.execute(
+        """SELECT * FROM policy_decisions
+            WHERE content_item_id = ? AND policy_id = ?""",
+        (content_item_id, policy_id),
+    ) as cur:
+        row = await cur.fetchone()
+    return _row_to_policy_decision(row) if row else None
+
+
+async def list_policy_decisions_for_content(
+    conn: aiosqlite.Connection,
+    content_item_id: int,
+) -> list[PolicyDecision]:
+    """Return all policy decisions recorded for a content item."""
+    conn.row_factory = aiosqlite.Row
+    async with conn.execute(
+        """SELECT * FROM policy_decisions
+            WHERE content_item_id = ?
+            ORDER BY evaluated_at, id""",

```

```

+         (content_item_id,),
+     ) as cur:
+         rows = await cur.fetchall()
+     return [_row_to_policy_decision(row) for row in rows]
+
+
+ # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --

@@ -1860,6 +1977,9 @@ __all__ = [
    "get_default_policy",
    "load_policy_config",
    "save_policy_config",
+   "record_policy_decision",
+   "get_policy_decision",
+   "list_policy_decisions_for_content",
    # Destination / Routing / Delivery repos (ADR 0011, Track C)
    "upsert_destination",
    "get_destination",
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index 70e2a8a..fd4c85e 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -296,6 +296,22 @@ class DeliveryAttempt(BaseModel):
    next_retry_at: datetime | None = None

+class PolicyDecision(BaseModel):
+    """Persisted ADR 0012 policy evaluation trace for one content item."""
+
+    model_config = ConfigDict(frozen=True)
+
+    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
+    content_item_id: int = Field(description="FK -> content_items.id")
+    policy_id: int = Field(description="FK -> policies.id")
+    passed: bool
+    trace: list[dict[str, object]] = Field(
+        default_factory=list,
+        description="Ordered StageResult trace serialized in policy_decisions.trace_json",
+    )
+    evaluated_at: datetime = Field(default_factory=_utcnow)
+
+class FilterConfig(BaseModel):
+    """Per-user filter configuration.

diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index ab0c029..b9ec862 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -834,6 +834,19 @@ async def evaluate_pipeline(
    return True, results

+def stage_results_to_trace(results: list[StageResult]) -> list[dict[str, object]]:
+    """Convert in-memory stage results to a JSON-serializable trace."""
+    return [
+        {
+            "stage_name": result.stage_name,
+            "passed": result.passed,
+            "reason": result.reason,
+            "metadata": result.metadata,
+        }
+        for result in results
+    ]

```

```

+
+
__all__ = [
    "CaptionSafetyConfig",
    "CaptionSafetyStage",
@@ -859,4 +872,5 @@ __all__ = [
    "parse_caption_safety_config",
    "parse_domain_reputation_config",
    "parse_topic_preferences",
+    "stage_results_to_trace",
]

```

6. assistant (2026-07-06T23:48:56.526Z)

Now the telethon_client.py and backfill.py diffs.

7. user (2026-07-06T23:48:58.650Z)

```

diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
index ff5d8a3..624d332 100644
--- a/src/tg_video_filter/backfill.py
+++ b/src/tg_video_filter/backfill.py
@@ -38,6 +38,7 @@ from tg_video_filter.telethon_client import (
    _extract_document_id,
    _message_text,
    _peer_id,
+    _record_policy_decision,
    extract_video_meta,
)

@@ -243,8 +244,11 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
    result = engine.evaluate(meta)
    matched = result.matched
    reasons = list(result.reasons)
+    policy_passed = True
+    policy_results = []
+    policy_id = None
    if matched and source is not None:
-        policy_passed, policy_results = await _evaluate_policy_stages(
+        policy_passed, policy_results, policy_id = await _evaluate_policy_stages(
            conn,
            user.id,
            source,
@@ -284,6 +288,13 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
    matched,
    reasons,
)
+    await _record_policy_decision(
+        conn,
+        inserted=inserted,
+        policy_id=policy_id,
+        policy_passed=policy_passed,
+        policy_results=policy_results,
+    )

    if matched:
        summary["videos_matched"] += 1
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index c5f760f..37089ae 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -52,6 +52,7 @@ from tg_video_filter.policy import (
    TopicPreferenceStage,

```

```

        UrlReputationStage,
        evaluate_pipeline,
+       stage_results_to_trace,
    )

    log = logging.getLogger(__name__)
@@ -279,8 +280,11 @@ def make_new_message_handler(
        result = engine_holder[ENGINE_KEY].evaluate(meta)
        matched = result.matched
        reasons = list(result.reasons)
+       policy_passed = True
+       policy_results: list[StageResult] = []
+       policy_id: int | None = None
        if matched and source is not None:
-           policy_passed, policy_results = await _evaluate_policy_stages(
+           policy_passed, policy_results, policy_id = await _evaluate_policy_stages(
                conn,
                user.id,
                source,
@@ -335,6 +339,13 @@ def make_new_message_handler(
        matched,
        reasons,
    )
+       await _record_policy_decision(
+           conn,
+           inserted=inserted,
+           policy_id=policy_id,
+           policy_passed=policy_passed,
+           policy_results=policy_results,
+       )

        if matched:
            # Build link text for LINK/BOTH modes (None for FORWARD).
@@ -433,14 +444,14 @@ async def _evaluate_policy_stages(
    account_id: int,
    source: Source,
    content_item: ContentItem,
- ) -> tuple[bool, list[StageResult]]:
+ ) -> tuple[bool, list[StageResult], int | None]:
    """Evaluate ADR 0012 policy stages for a candidate item."""
    policy = await db.get_default_policy(conn, account_id)
    if policy is None:
-         return True, []
+         return True, [], None
    # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
    # user-facing enable command must land with fetcher/budget wiring.
-         return await evaluate_pipeline(
+         passed, results = await evaluate_pipeline(
            PolicyContext(content_item=content_item, source=source, policy=policy),
            [
                TopicPreferenceStage(),
@@ -449,6 +460,27 @@ async def _evaluate_policy_stages(
            DeepInspectionStage(),
        ],
    )
+       return passed, results, policy.id
+
+
+async def _record_policy_decision(
+    conn: aiosqlite.Connection,
+    *,
+    inserted: VideoRecord,
+    policy_id: int | None,
+    policy_passed: bool,
+    policy_results: list[StageResult],

```

```

+) -> None:
+     """Persist a policy trace after the content item has been inserted."""
+     if not policy_results or policy_id is None or inserted.id is None:
+         return
+     await db.record_policy_decision(
+         conn,
+         content_item_id=inserted.id,
+         policy_id=policy_id,
+         passed=policy_passed,
+         trace=stage_results_to_trace(policy_results),
+     )

```

```

def _peer_id(peer: object) -> int:
@@ -485,5 +517,6 @@ __all__ = [
    "make_new_message_handler",
    "_content_item_preview",
    "_evaluate_policy_stages",
+   "_record_policy_decision",
    "_message_text",
]

```

8. assistant (2026-07-06T23:49:01.642Z)

Now let me examine the StageResult metadata types, to assess the serialization crash scenario. Let me look at what the stages put into metadata.

9. user (2026-07-06T23:49:03.583Z)

```

48:     def __init__(self, exceeded_budget: str, **metadata: object) -> None:
51:         self.metadata = metadata
145:     metadata: dict[str, object] = field(default_factory=dict)
148:     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
153:         metadata=metadata,
157:     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
162:         metadata=metadata,
529:         metadata = _url_reputation_metadata((), config)
534:         **metadata,
545:         **_url_reputation_metadata(urls, config),
554:         **_url_reputation_metadata(urls, config),
563:         **_url_reputation_metadata(urls, config),
567: def _url_reputation_metadata(
572:     metadata: dict[str, object] = {
578:         metadata["ignored_invalid_domains"] = list(config.invalid_domains)
579:     return metadata
602:         metadata = _caption_safety_metadata(config, caption_present=False)
605:         "caption safety has no valid active checks policy_id=%s metadata=%s",
607:         metadata,
612:         **metadata,
631:         **_caption_safety_metadata(config, caption_present=False),
642:         **_caption_safety_metadata(
673:         **_caption_safety_metadata(
686:         **_caption_safety_metadata(
695: def _caption_safety_metadata(
702:     metadata: dict[str, object] = {
708:         metadata["caption_truncated"] = True
709:         metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
711:         metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
713:         metadata["ignored_caption_term_count"] = config.ignored_term_count
715:         metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
717:         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
718:     return metadata
763:         **exc.metadata,
844:         "metadata": result.metadata,

```

10. assistant (2026-07-06T23:49:06.489Z)

Let me look at the metadata builders more closely, particularly the URL reputation one that may include non-primitive values, and the DeepInspectionStage / budget metadata.

11. user (2026-07-06T23:49:07.105Z)

```
560         return StageResult.pass_(
561             self.stage_name,
562             "links passed domain reputation checks",
563             **_url_reputation_metadata(urls, config),
564         )
565
566
567     def _url_reputation_metadata(
568         urls: tuple[ExtractedUrl, ...],
569         config: DomainReputationConfig,
570     ) -> dict[str, object]:
571         domains = sorted({url.domain for url in urls})
572         metadata: dict[str, object] = {
573             "link_count": len(urls),
574             "domains": domains,
575             "blocked_domain_count": len(config.blocked_domains),
576         }
577         if config.invalid_domains:
578             metadata["ignored_invalid_domains"] = list(config.invalid_domains)
579         return metadata
580
581
582     class CaptionSafetyStage:
583         """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
584
585         stage_name = "caption-safety"
586
587         async def evaluate(self, ctx: PolicyContext) -> StageResult:
588             try:
589                 config = load_caption_safety_config(ctx.policy)
590             except (json.JSONDecodeError, ValueError) as exc:
591                 log.warning(
592                     "caption safety disabled due to invalid config policy_id=%s",
593                     ctx.policy.id,
594                     exc_info=True,
595                 )
596             return StageResult.pass_(
597                 self.stage_name,
598                 f"invalid caption safety config ignored: {exc}",
599                 invalid_caption_safety_config=True,
```

12. assistant (2026-07-06T23:49:12.398Z)

The current F1-F5 metadata is all JSON-safe primitives/lists. The serialization crash scenario is about a **future** stage returning non-JSON metadata. That's a latent risk noted in the prompt. Let me confirm the crash propagation path is real by reproducing, and check ``_now`/`_utcnow`` for tz-awareness. Let me look at db ``_now``.

13. user (2026-07-06T23:49:14.383Z)

```
src/tg_video_filter/db.py:807:def _now() -> datetime:
src/tg_video_filter/db.py:1382:         created_at=(existing.created_at if existing is not
None else _now()),
src/tg_video_filter/db.py:1383:         updated_at=_now(),
src/tg_video_filter/db.py:1406:     evaluated_at = evaluated_at or _now()
src/tg_video_filter/models.py:17:def _utcnow() -> datetime:
src/tg_video_filter/models.py:53:     created_at: datetime = Field(default_factory=_utcnow)
```

```

src/tg_video_filter/models.py:74: created_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:93: created_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:110: seen_first_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:145: seen_first_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:183: seen_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:207: created_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:208: updated_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:242: created_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:265: created_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:294: created_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:312: evaluated_at: datetime = Field(default_factory=_utcnow)
src/tg_video_filter/models.py:428: seen_at: datetime = Field(default_factory=_utcnow)

```

14. user (2026-07-06T23:49:17.622Z)

```

807     def _now() -> datetime:
808         """Timezone-aware UTC 'now', used for policy timestamps."""
809         return datetime.now(UTC)
810

```

15. assistant (2026-07-06T23:49:21.432Z)

tz-aware confirmed. Now let me verify: does `record\_policy\_decision` call `conn.commit()`? Yes it does (line ~1416). Note the `on_new_message` handler ? does it commit its own transaction, or does `record_policy_decision`'s commit interfere with an in-flight transaction? Let me check the transaction handling around `insert_video` and `record_policy_decision` in the handler. This matters: `record_policy_decision` calls `conn.commit()` unconditionally.

Let me look at the `on_new_message` flow around `insert_video` and delivery to understand transaction boundaries.

16. user (2026-07-06T23:49:22.480Z)

```

320         except sqlite3.IntegrityError:
321             # Lost a dedup race: a concurrent delivery of the same file
322             # (same document_id or same message instance) won the insert.
323             # The three-tier check above already classified this as new, so
324             # the collision means another task is processing (or has
325             # processed) it ? log and skip.
326             log.info(
327                 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
328                 user.id,
329                 channel_id,
330                 message_id,
331                 document_id,
332             )
333             return
334         log.info(
335             "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
336             user.id,
337             channel_id,
338             message_id,
339             matched,
340             reasons,
341         )
342         await _record_policy_decision(
343             conn,
344             inserted=inserted,
345             policy_id=policy_id,
346             policy_passed=policy_passed,
347             policy_results=policy_results,
348         )
349
350     if matched:

```

```

351         # Build link text for LINK/BOTH modes (None for FORWARD).
352         link_text = None
353         if delivery_mode != DeliveryMode.FORWARD:
354             channel_title = _channel_title(event)
355             link_text = build_link_text(
356                 message_link=getattr(message, "link", None),
357                 duration_s=meta.duration_s,
358                 height=meta.height,
359                 size_bytes=meta.size_bytes,
360                 channel_title=channel_title,
361             )
362         await deliver(
363             client,
364             message,
365             conn,
366             inserted.id,
367             delivery_mode,
368             link_text=link_text,
369             target=delivery_target,
370         )
371
372     return on_new_message
373
374
375 # --- helpers -----
376
377
378 def _extract_document_id(message: Message) -> int | None:
379     """Extract Telegram's ``document.id`` from a video message.
380
381     Returns None if the message has no document media or the id is absent.
382     This is the most reliable content-based dedup key: Telegram reuses the
383     same ``document.id`` when a file is forwarded across channels.
384     """
385     media = getattr(message, "media", None)
386     if isinstance(media, MessageMediaDocument):
387         media = media.document
388     if isinstance(media, Document):
389         return getattr(media, "id", None)
390     return None
391
392
393 def _message_text(message: object) -> str | None:
394     """Return caption/message text normalized for storage, or None if empty."""

```

17. assistant (2026-07-06T23:49:25.355Z)

Let me look at how `insert\_video` handles its transaction/commit, to see whether `record\_policy\_decision`'s unconditional `conn.commit()` could commit a partial delivery transaction prematurely, and whether the flow uses a single shared connection.

18. user (2026-07-06T23:49:27.723Z)

```

232:     await conn.commit()
1000:    await conn.commit()
1043:    await conn.commit()
1186:    await conn.commit()
1249:    await conn.commit()
1316:    await conn.commit()
1424:    await conn.commit()
1485:    await conn.commit()
1548:    await conn.commit()
1726:    await conn.commit()
1773:async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:

```

19. user (2026-07-06T23:49:30.154Z)

```
1773     async def insert_video(conn: aioredis.Connection, video: VideoRecord) -> VideoRecord:
1774         """Deprecated shim: insert into ``content_items`` (ADR 0011).
1775
1776         Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1777         if absent), computes the provider-neutral ``dedup_key`` via
1778         :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1779         ``VideoRecord`` shape is reconstructed from the inserted row.
1780         """
1781         src = await upsert_source(
1782             conn,
1783             Source(
1784                 account_id=video.user_id,
1785                 provider="telegram",
1786                 provider_source_id=str(video.channel_id),
1787                 kind="channel",
1788                 watched=False,
1789             ),
1790         )
1791         assert src.id is not None # upsert_source populates the surrogate PK
1792         item = ContentItem(
1793             account_id=video.user_id,
1794             source_id=src.id,
1795             provider="telegram",
1796             provider_message_id=str(video.message_id),
1797             content_type="video",
1798             text=video.text,
1799             document_id=str(video.document_id) if video.document_id is not None else None,
1800             duration_s=video.duration_s,
1801             width=video.width,
1802             height=video.height,
1803             size_bytes=video.size_bytes,
1804             mime_type=video.mime_type,
1805             dedup_key=compute_dedup_key(
1806                 provider="telegram",
1807                 document_id=(str(video.document_id) if video.document_id is not None else
1808                             None),
1809                 source_id=src.id,
1810                 provider_message_id=str(video.message_id),
1811                 size_bytes=video.size_bytes,
1812                 duration_s=video.duration_s,
1813                 height=video.height,
```

20. assistant (2026-07-06T23:49:34.178Z)

insert_video commits internally (aioredis in autocommit-ish mode with explicit commits per repo). Each repo function commits its own writes, so there's no long-lived multi-statement transaction that record_policy_decision's commit would prematurely flush. This is the established pattern in this codebase. Not a defect.

Now let me run the key repros. First, the serialization crash with non-serializable metadata, and confirm it propagates uncaught. Let me write a repro.

21. assistant (2026-07-06T23:49:41.579Z)

```
import asyncio, aioredis, tempfile, os
from datetime import datetime, timezone
from tg_video_filter import db
from tg_video_filter.policy import stage_results_to_trace, StageResult

async def main():
    # 1) Non-serializable metadata via stage_results_to_trace -> json.dumps
    sr = StageResult.pass_("weird-stage", "ok", obj=object(), when=datetime.now(timezone.utc))
    trace = stage_results_to_trace([sr])
```

```

print("trace built:", type(trace), trace)
try:
    import json
    json.dumps(trace, separators=(",", ":"), sort_keys=True)
    print("JSON OK (unexpected)")
except TypeError as e:
    print("json.dumps TypeError:", e)

# 2) Confirm db.record_policy_decision propagates the TypeError uncaught
fd, path = tempfile.mkstemp(suffix=".db"); os.close(fd)
try:
    conn = await aiosqlite.connect(path)
    await db.apply_migrations(conn)
    # need a content_item + policy to satisfy FK? FK enforcement off by default in sqlite
    try:
        await db.record_policy_decision(
            conn, content_item_id=1, policy_id=1, passed=True, trace=trace,
        )
        print("record_policy_decision OK (unexpected)")
    except TypeError as e:
        print("record_policy_decision raised TypeError (propagates uncaught):", e)
    await conn.close()
finally:
    os.unlink(path)

asyncio.run(main())

```

22. user (2026-07-06T23:49:41.611Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_serial.py
(file state is current in your context ? no need to Read it back)

23. user (2026-07-06T23:51:45.654Z)

Command running in background with ID: bkefo1529. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bkefo1529.output. You will be notified when it completes. To check interim output, use Read on that file path.

24. user (2026-07-06T23:51:52.273Z)

```

trace built: <class 'list'> [{'stage_name': 'weird-stage', 'passed': True, 'reason': 'ok',
'metadata': {'obj': <object object at 0x0000026D85150E90>, 'when': datetime.datetime(2026, 7, 6,
23, 49, 46, 474963, tzinfo=datetime.timezone.utc)}}]
json.dumps TypeError: Object of type object is not JSON serializable
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_serial.py", line 22, in main
    await db.apply_migrations(conn)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'tg_video_filter.db' has no attribute 'apply_migrations'

```

During handling of the above exception, another exception occurred:

```

Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_serial.py", line 35, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~

```

```
Exit code 143
Command timed out after 2m 0s
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro2.py", line 19, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro2.py", line 11, in main
    await db._apply_migrations(conn)
  File "C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py", line 339, in _apply_migrations
    await fn(conn)
  File "C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py", line 358, in migration_001_add_document_id
    await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
```

32. assistant (2026-07-06T23:54:29.851Z)

Both confirmed:

```
- **A)** Non-serializable metadata propagates `TypeError` uncaught out of
`db.record_policy_decision`.
- **B)** FK is enforced (`PRAGMA foreign_keys = 1`). `record_policy_decision` raises
`IntegrityError` if `content_item_id/policy_id` don't exist.
```

Now B is interesting. In the wiring, `\_record\_policy\_decision` is called AFTER `insert\_video`, so the `content\_item` exists. But `policy\_id` comes from `policy.id` returned by `\_evaluate\_policy\_stages`. Is there any path where `policy\_id` points to a policy that has since been deleted, or where the policy row's FK is stale? More important: the guard checks `inserted.id is None` but the `content\_item\_id` is `inserted.id`. Let me verify `inserted` (a `VideoRecord`) has `.id` set to the `content\_item` id. Let me check what `insert\_video` returns.

33. user (2026-07-06T23:54:30.414Z)

```
1813         width=video.width,
1814     ),
1815     matched=video.matched,
1816     seen_at=video.seen_at,
1817 )
1818 item = await insert_content_item(conn, item)
1819 return _content_item_to_video(item, src, video.user_id, video.channel_id)
1820
1821
1822 async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
1823     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1824
1825     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1826     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1827     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1828     delivery still targets the single ``delivery_target`` and retried delivery
1829     is harmless (forwards are idempotent). This shim is retained so existing
1830     call sites in ``delivery.py`` keep working without a flag day.
1831     """
1832     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1833     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1834     return None
1835
1836
1837 def _content_item_to_video(
1838     item: ContentItem, src: Source, user_id: int, channel_id: int
1839 ) -> VideoRecord:
1840     """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1841
1842     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1843     per-destination delivery state moves to Track C). Existing callers read it
1844     only to decide whether to skip re-delivery; since forwards are idempotent
1845     the ``False`` default is safe for Phase 1.
1846     """
1847     return VideoRecord(
1848         id=item.id,
1849         user_id=user_id,
1850         channel_id=channel_id,
1851         message_id=_safe_int_cast(item.provider_message_id, "provider_message_id"),
1852         text=item.text,
1853         document_id=(
1854             _safe_int_cast(item.document_id, "document_id")
1855             if item.document_id is not None
1856             else None
1857         ),
1858         duration_s=item.duration_s,
1859         width=item.width,
1860         height=item.height,
1861         size_bytes=item.size_bytes,
1862         mime_type=item.mime_type,
```

```

1863         matched=item.matched,
1864         forwarded=False,
1865         seen_at=item.seen_at,
1866     )
1867
1868
1869     # --- filter config shims (backwards-compatible view over policies) -----
1870
1871
1872     async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:

```

34. assistant (2026-07-06T23:54:34.358Z)

`inserted.id = item.id` which is the content_items PK. Good. So the FK content_item_id is valid after insert_video.

Now the key latent question on the write path: **the serialization crash is uncaught and on_new_message has no try/except around `\_record\_policy\_decision`.** Let me verify there's truly no surrounding try/except in on_new_message that would catch it. Let me look at the full handler structure from the top.

35. user (2026-07-06T23:54:35.943Z)

```

250         "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
251         "channel=%s msg=%s original_channel=%s original_msg=%s",
252         user.id,
253         meta.size_bytes,
254         meta.duration_s,
255         meta.height,
256         meta.width,
257         channel_id,
258         message_id,
259         existing.channel_id,
260         existing.message_id,
261     )
262     return
263
264     source = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))
265
266     # Lazily record the channel (best-effort; failures are non-fatal).
267     try:
268         channel = await db.upsert_channel(
269             conn,
270             Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
271         )
272         if channel.id is not None:
273             source = await db.get_source_by_id(conn, channel.id)
274     except Exception:
275         log.warning(
276             "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
277         )
278
279     text = _message_text(message)
280     result = engine_holder[ENGINE_KEY].evaluate(meta)
281     matched = result.matched
282     reasons = list(result.reasons)
283     policy_passed = True
284     policy_results: list[StageResult] = []
285     policy_id: int | None = None
286     if matched and source is not None:
287         policy_passed, policy_results, policy_id = await _evaluate_policy_stages(
288             conn,

```

```

289         user.id,
290         source,
291         _content_item_preview(
292             user.id,
293             source,
294             message_id,
295             document_id,
296             meta,
297             text,
298             matched,
299         ),
300     )
301     if policy_results:
302         reasons.extend(result.reason for result in policy_results)
303         matched = policy_passed
304
305     record = VideoRecord(
306         user_id=user.id,
307         channel_id=channel_id,
308         message_id=message_id,
309         text=text,
310         document_id=document_id,
311         duration_s=meta.duration_s,
312         width=meta.width,
313         height=meta.height,
314         size_bytes=meta.size_bytes,
315         mime_type=meta.mime_type,
316         matched=matched,
317     )
318     try:
319         inserted = await db.insert_video(conn, record)
320     except sqlite3.IntegrityError:
321         # Lost a dedup race: a concurrent delivery of the same file

```

36. assistant (2026-07-06T23:54:43.646Z)

Confirmed: `\_record\_policy\_decision` at line 342 is outside any try/except. A serialization crash there would propagate out of the handler.

Now the important scoping question for severity: is this a defect ****in THIS PR's diff**** or a hypothetical future risk? Today, all F1-F5 stage metadata is JSON-safe (confirmed by reading the metadata builders ? all str/int/list-of-str). The prompt itself flags this as a ***future*** ContentFetcher risk. So the current diff does not crash with current stages. This is a ****latent/defensive**** finding: the write path has no serialization guard, so any future stage adding non-serializable metadata crashes ingestion. Same class as the F5 fetcher-error lesson. That's a low/medium note, not a currently-triggering bug.

Let me verify there is genuinely no non-serializable metadata reachable today. The `when=datetime` case: do any current stages put a datetime or set or tuple in metadata? Let me grep for metadata values that could be non-JSON (sets, tuples, datetimes).

37. user (2026-07-06T23:54:45.745Z)

```

149:         return StageResult(
158:         return StageResult(
385:             return StageResult.pass_(
391:             return StageResult.pass_(self.stage_name, "topic preferences disabled")
396:             return StageResult.fail(
403:             return StageResult.pass_(self.stage_name, "no include topics configured")
408:             return StageResult.pass_(
413:             return StageResult.fail(self.stage_name, "no include topics matched")
417:             return StageResult.fail(
423:         return StageResult.pass_(
516:             return StageResult.pass_(
531:             return StageResult.pass_(

```

```

536:         return StageResult.pass_(self.stage_name, "domain reputation disabled")
542:         return StageResult.pass_(
551:             return StageResult.fail(
560: return StageResult.pass_(
596:     return StageResult.pass_(
609:         return StageResult.pass_(
614:         return StageResult.pass_(self.stage_name, "caption safety disabled")
628:         return StageResult.pass_(
639:             return StageResult.fail(
670:             return StageResult.fail(
683: return StageResult.pass_(
735:     return StageResult.pass_(
741:     return StageResult.pass_(self.stage_name, "deep inspection disabled")
748:     return StageResult.fail(
758:     return StageResult.fail(
771:     return StageResult.fail(
782: return StageResult.fail(
788: return StageResult.fail(
798: return StageResult.fail(
805: return StageResult.fail(
812: return StageResult.fail(

```

38. assistant (2026-07-06T23:54:49.268Z)

Let me spot-check the metadata kwargs at a few of these ? particularly topic preference (matched topics could be sets) and the DeepInspection budget metadata.

39. user (2026-07-06T23:54:49.981Z)

```

385         return StageResult.pass_(
386             self.stage_name,
387             f"invalid topic config ignored: {exc}",
388             invalid_topic_config=True,
389         )
390     if not prefs.enabled:
391         return StageResult.pass_(self.stage_name, "topic preferences disabled")
392
393     haystack = _topic_haystack(ctx)
394     matched_excludes = _matched_topics(prefs.exclude, haystack)
395     if matched_excludes:
396         return StageResult.fail(
397             self.stage_name,
398             f"excluded topic matched: {matched_excludes[0]}",
399             matched_exclude_topics=list(matched_excludes),
400         )
401
402     if not prefs.include:
403         return StageResult.pass_(self.stage_name, "no include topics configured")
404
405     matched_includes = _matched_topics(prefs.include, haystack)
406     if prefs.mode == "any":
407         if matched_includes:
408             return StageResult.pass_(
409                 self.stage_name,
410                 f"matched include topic: {matched_includes[0]}",
411                 matched_include_topics=list(matched_includes),
412             )
413         return StageResult.fail(self.stage_name, "no include topics matched")
414
415     missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
416     if missing:
417         return StageResult.fail(
418             self.stage_name,
419             f"missing required topic: {missing[0]}",

```

```

420             matched_include_topics=list(matched_includes),
421             missing_include_topics=list(missing),
422         )
423     return StageResult.pass_(
424         self.stage_name,

```

40. assistant (2026-07-06T23:54:54.497Z)

All current metadata is explicitly wrapped with `list(...)` and primitives ? JSON-safe today. So the serialization crash is confirmed as latent/defensive only, not currently triggerable. Low severity note.

Now let me look at the DeepInspection budget metadata (the `BudgetExceeded` exception path around line 763 that does `\*\*exc.metadata`), since that's the F5-adjacent path most likely to grow non-JSON metadata.

41. user (2026-07-06T23:54:55.304Z)

```

744         if budget_result is not None:
745             return budget_result
746
747         if ctx.fetcher is None:
748             return StageResult.fail(
749                 self.stage_name,
750                 "deep inspection fetcher missing",
751                 missing_fetcher=True,
752             )
753
754         assert ctx.budget is not None
755         try:
756             return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
757         except PolicyBudgetExceeded as exc:
758             return StageResult.fail(
759                 self.stage_name,
760                 "deep inspection budget exceeded",
761                 budget_exceeded=True,
762                 exceeded_budget=exc.exceeded_budget,
763                 **exc.metadata,
764             )
765         except Exception as exc:
766             log.warning(
767                 "deep inspection fetcher failed policy_id=%s",
768                 ctx.policy.id,
769                 exc_info=True,
770             )
771             return StageResult.fail(
772                 self.stage_name,
773                 "deep inspection fetcher failed",
774                 fetcher_error_type=type(exc).__name__,
775                 fetcher_error=str(exc),
776             )
777
778
779     def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
780         budget = ctx.budget
781         if budget is None:
782             return StageResult.fail(
783                 DeepInspectionStage.stage_name,
784                 "deep inspection budget missing",
785                 missing_budget=True,
786             )
787         if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
788         <= 0:
789             return StageResult.fail(
790                 DeepInspectionStage.stage_name,

```

```

790         "deep inspection budget invalid",
791         invalid_budget=True,
792         max_bytes_per_item=budget.max_bytes_per_item,
793         timeout_s=budget.timeout_s,
794         max_concurrent=budget.max_concurrent,
795     )
796     size_bytes = ctx.content_item.size_bytes
797     if size_bytes is None:
798         return StageResult.fail(
799             DeepInspectionStage.stage_name,
800             "deep inspection budget cannot be enforced without content size",
801             missing_size_bytes=True,
802             max_bytes_per_item=budget.max_bytes_per_item,
803         )

```

42. assistant (2026-07-06T23:55:04.217Z)

Line 763 ``**exc.metadata` where `PolicyBudgetExceeded(exceeded_budget, **metadata)` ? `exc.metadata` is arbitrary caller-supplied. This is the concrete near-term path where a future ContentFetcher (F5) could pass non-JSON metadata into the budget exception, which then flows into the trace and crashes `record_policy_decision`. This reinforces the finding but doesn't change that it's not triggerable in the current diff.`

Now let me examine the ``_row_to_policy_decision`` "must decode to a JSON array" raise. Can our own writes ever produce non-list trace_json? ``_policy_trace_json`` does ``json.dumps(trace, ...)`` where trace is ``list[dict]`` from ``stage_results_to_trace``, always a list. So from our own writes it's always an array. That ValueError only fires on externally-corrupted DB rows. Not a defect from the diff.

Now let me check the UPSERT idempotency and ``evaluated_at`` ? already verified by main reviewer, but let me confirm the trace serialization is stable and check one subtle thing: ``sort_keys=True`` sorts dict keys, which reorders the stage_name/passed/reason/metadata keys within each dict ? but the ``**list order`` (stage order) is preserved since ``json.dumps`` doesn't sort list elements. Good, stage order preserved.

Let me also confirm there's no double-commit / connection issue and verify the full happy-path persist+UPSERT works end to end with a real policy and content item.

43. user (2026-07-06T23:55:06.164Z)

```

48:     def __init__(self, exceeded_budget: str, **metadata: object) -> None:
139: class StageResult:
142:     passed: bool
143:     reason: str
144:     stage_name: str
145:     metadata: dict[str, object] = field(default_factory=dict)
148:     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
157:     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
572:     metadata: dict[str, object] = {
702:     metadata: dict[str, object] = {

```

44. assistant (2026-07-06T23:55:17.943Z)

Now let me verify the guard's behavior against the FK-crash risk more carefully. The guard is ``if not policy_results or policy_id is None or inserted.id is None: return``. Consider: ``matched=True``, source present, a default policy exists (policy_id set), stages ran (policy_results non-empty). Then insert_video runs. ``**What if insert_video hit the IntegrityError dedup-race path?`` In that case the handler ``return`s` at line 333 before reaching ``_record_policy_decision`` ? so no orphan record. Good.

But there's a subtler FK concern: could ``policy_id`` reference a policy that gets deleted between evaluation and record? In a single-threaded per-user handler with one connection this is unlikely, and would only raise a caught-nowhere IntegrityError ? same latent class. Not in-diff triggerable.

Let me now check one concrete thing about the guard interacting with `matched`: when policy fails (`policy\_passed=False`), `matched` becomes False (line 303), item is inserted with matched=False, no delivery. But `policy\_results` is non-empty and `policy\_id` set, so `\_record\_policy\_decision` DOES record passed=False. Let me confirm no delivery_attempt is created for rejected. Delivery only happens `if matched:` (line 350), and matched is False. Confirmed: rejected item ? decision row passed=False, no delivery. Correct behavior.

Let me run the existing test suite quickly for these files to confirm nothing in the diff is broken, and check test coverage of the guard branches.

45. user (2026-07-06T23:55:24.849Z)

src\tg_video_filter\bootstrap.py	23	23	0	0	0%	17-78
src\tg_video_filter\command_service.py	218	177	94	0	13%	53-61, 66-71, 78-94, 98-107, 112-126, 131, 215-343, 351-361, 367-374, 378-391, 395-406, 416-431, 449-478
src\tg_video_filter\commands.py	46	30	12	0	28%	58-88, 111-113, 128-131
src\tg_video_filter\config.py	26	5	2	0	75%	62-64, 69, 79
src\tg_video_filter\db.py	380	31	58	10	90%	535, 664-667, 698, 701, 928, 953, 1347, 1365, 1591-1592, 1604, 1646, 1722-1726, 1733-1741, 1759-1769, 1900-1908, 2007
src\tg_video_filter\delivery.py	102	80	48	0	15%	40-48, 53-58, 63-65, 87-95, 107-121, 145-160, 177-192, 218-278
src\tg_video_filter\filters__init__.py	39	26	18	0	23%	32-33, 42-52, 62-75
src\tg_video_filter\filters\base.py	26	2	0	0	92%	48, 52
src\tg_video_filter\filters\content_type.py	23	15	12	0	23%	18-22, 29-33, 40-44
src\tg_video_filter\filters\duration.py	15	10	6	0	24%	17-18, 21-28
src\tg_video_filter\filters\resolution.py	13	8	6	0	26%	16, 21-27
src\tg_video_filter\filters\size.py	18	11	6	0	29%	18, 21-29, 34
src\tg_video_filter\manager.py	36	36	8	0	0%	26-204
src\tg_video_filter\models.py	171	1	8	1	99%	380
src\tg_video_filter\policy.py	415	35	152	25	89%	69->exit, 75, 83, 96, 99-100, 124, 171, 232, 257, 294, 321, 329, 337, 345, 353, 364, 366->362, 403, 413, 443, 451, 455, 464-465, 482, 510-516, 531, 604-609, 713, 715, 729-735
src\tg_video_filter\telethon_client.py	84	47	34	3	34%	82-120, 372, 385-390, 395-401, 414-415, 451, 476, 494-499, 511->exit

TOTAL	1692	593	480	39	59%	

FAIL Required test coverage of 80.0% not reached. Total coverage: 59.02%
132 passed in 3.70s

46. assistant (2026-07-06T23:55:29.213Z)

132 passed (the coverage fail is just because I ran a subset). Note `telethon\_client.py` line 451 and 476 (the `\_record\_policy\_decision` guard/body region) ? let me check what's covered. Actually lines 464-499 region includes `\_record\_policy\_decision`. Coverage shows 476 uncovered. Let me check whether the guard's early-return branch and the FK-crash body have direct tests. Let me look at what test_policy added.

47. user (2026-07-06T23:55:31.965Z)

```

5:@@ -65,6 +65,7 @@ async def test_init_db_creates_tables(tmp_path: object) -> None:
13:@@ -694,6 +695,14 @@ async def test_fresh_db_user_version_is_latest(conn:
aiosqlite.Connection) -> No
17:+async def test_migration_005_policy_decisions_is_idempotent(
25: async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
28:@@ -1205,6 +1214,124 @@ async def test_policy_upsert_and_default_lookup(conn:
aiosqlite.Connection) -> N
65:+async def test_record_policy_decision_accepted_trace(conn: aiosqlite.Connection) -> None:
78:+     decision = await db.record_policy_decision(
94:+async def test_record_policy_decision_rejected_without_delivery_attempt(
109:+     decision = await db.record_policy_decision(

```

```

123:+async def test_record_policy_decision_upserts_trace(conn: aiosqlite.Connection) -> None:
128:+    await db.record_policy_decision(
135:+    updated = await db.record_policy_decision(
150: async def test_filter_config_shim_uses_default_policy(conn: aiosqlite.Connection) -> None:
171:+from tg_video_filter.telethon_client import _evaluate_policy_stages,
_record_policy_decision
175:@@ -817,7 +817,7 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:
184:@@ -826,10 +826,11 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:
197:@@ -837,11 +838,12 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:
211:@@ -849,13 +851,14 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:
227:@@ -863,6 +866,7 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:
235:@@ -878,3 +882,54 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:
241:+async def test_record_policy_decision_helper_persists_stage_trace() -> None:
269:+    await _record_policy_decision(

```

48. assistant (2026-07-06T23:55:39.349Z)

There's a test for the `\_record\_policy\_decision` helper. Let me check whether it covers the guard's early-return (empty policy_results) and importantly whether there's any test where non-serializable metadata is passed (there won't be). Let me look at that helper test.

49. user (2026-07-06T23:55:41.130Z)

(Bash completed with no output)

50. user (2026-07-06T23:55:44.665Z)

```

    assert caption_passed is False
+    assert caption_policy_id == policy_id
    assert [result.stage_name for result in caption_rejected] == [
        "topic-preferences",
        "url-reputation",
@@ -878,3 +882,54 @@ async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
    }
    finally:
        await conn.close()
+
+
+async def test_record_policy_decision_helper_persists_stage_trace() -> None:
+    conn = await aiosqlite.connect(":memory:")
+    try:
+        await db._apply_schema(conn)
+        await db.upsert_user(conn, db.User(id=111, phone="+15555550100")) # type: ignore[attr-
defined]
+        policy = await db.upsert_policy(
+            conn,
+            Policy(account_id=111, name="default", config_json={}, is_default=True),
+        )
+        assert policy.id is not None
+        inserted = await db.insert_video(
+            conn,
+            VideoRecord(
+                user_id=111,
+                channel_id=-1001,
+                message_id=42,
+                matched=False,
+            ),
+        )
+

```

```

+         assert inserted.id is not None
+         results = [
+             StageResult.fail(
+                 "url-reputation",
+                 "blocked domain matched: bad.example",
+                 blocked_domain="bad.example",
+             )
+         ]
+
+         await _record_policy_decision(
+             conn,
+             inserted=inserted,
+             policy_id=policy.id,
+             policy_passed=False,
+             policy_results=results,
+         )
+
+         decision = await db.get_policy_decision(conn, inserted.id, policy.id)
+         assert decision is not None
+         assert decision.passed is False
+         assert decision.trace == [
+             {
+                 "stage_name": "url-reputation",
+                 "passed": False,
+                 "reason": "blocked domain matched: bad.example",
+                 "metadata": {"blocked_domain": "bad.example"},
+             }
+         ]
+     finally:
+         await conn.close()

```

51. assistant (2026-07-06T23:55:57.503Z)

Good coverage of the happy path with a rejected item. The guard's early-return branches and the non-serializable path are untested but that's a coverage note, not a bug.

I've now thoroughly examined the write path. Let me consolidate findings. Let me double-check one more potential real issue: the `list[dict[str, object]]` default in `_record_policy_decision` ? the guard in `_record_policy_decision` returns when `not policy_results`, but `db.record_policy_decision` itself accepts an empty trace and would write `[]`. Is there any caller passing an empty list directly? Only the wired helper, which guards against it. The `db` function with empty trace writes `[]` which round-trips fine (`_row_to_policy_decision` requires a list ? `[]` is a list). No issue.

Let me verify the `evaluated_at` naive-vs-aware and default: `_record_policy_decision` uses `evaluated_at` or `_now()` where `_now()` returns tz-aware; stored via `.isoformat()`. Confirmed tz-aware ISO, not naive `CURRENT_TIMESTAMP`.

Let me do a final check: is there any issue with `int(passed)` when `passed` is already a Python bool vs the model's `passed: bool`? `int(True)=1`, fine. And `_row_to_policy_decision` does `bool(row["passed"])`. Round-trips correctly.

Now let me assess whether the serialization crash rises to "medium." The prompt's severity rubric: medium = "real latent bug/FK/serialization crash." The serialization crash is explicitly called a latent serialization crash in the rubric's medium band. It is real (reproduced), it's latent (needs a future non-JSON metadata stage), and it's uncaught into ingestion. I'll rank it medium given the rubric explicitly lists "serialization crash" under medium, while noting it is not triggerable by current F1-F5 stages.

Let me verify the backfill path guard symmetry ? backfill initializes `policy_results = []` (untyped) and calls `_record_policy_decision` unconditionally, relying on the guard. Same as handler. And backfill's `_record_policy_decision` is also outside try/except? Let me check.

52. user (2026-07-06T23:55:58.491Z)

```

278             size_bytes=meta.size_bytes,
279             mime_type=meta.mime_type,
280             matched=matched,
281         )
282         inserted = await db.insert_video(conn, record)
283         log.info(
284             "backfill_video_seen user=%s channel=%s msg=%s matched=%s
reasons=%s",
285             user.id,
286             channel_id,
287             message_id,
288             matched,
289             reasons,
290         )
291         await _record_policy_decision(
292             conn,
293             inserted=inserted,
294             policy_id=policy_id,
295             policy_passed=policy_passed,
296             policy_results=policy_results,
297         )
298
299         if matched:
300             summary["videos_matched"] += 1
301             dialog_matched += 1
302             link_text = build_link_text(
303                 message_link=getattr(message, "link", None),
304                 duration_s=meta.duration_s,
305                 height=meta.height,
306                 size_bytes=meta.size_bytes,
307                 channel_title=title, # from the dialog object

```

53. assistant (2026-07-06T23:56:00.960Z)

Let me check whether backfill wraps the per-message loop in a try/except (which would contain a serialization crash to one message rather than aborting the whole backfill).

54. user (2026-07-06T23:56:02.762Z)

```

147:         try:
151:             async for message in client.iter_messages(
228:                 try:
235:                     except Exception:
327:             except FloodWaitError as e:
364:                 try:
366:                     except Exception:
402:                     try:
404:                         except ValueError as e:
444:                 try:
460:                     except asyncio.CancelledError:
464:                     except Exception:
465:                         log.exception("backfill_crashed user=%s days=%d", user.id, days)

```

55. user (2026-07-06T23:56:05.195Z)

```

147         try:
148             # NOTE: we do NOT use offset_date because it gets messages OLDER
149             # than the given date. Instead we iterate newest-first and manually
150             # break when we cross the cutoff boundary.
151             async for message in client.iter_messages(
152                 dialog.entity,
153                 wait_time=_MIN_WAIT_BETWEEN_BATCHES,
154                 reverse=False, # newest-first
155             ):

```

```

156         # Stop when we reach messages older than the cutoff.
157         msg_date = getattr(message, "date", None)
158         if msg_date is not None and msg_date < cutoff:
159             break
160
161         dialog_total += 1
162         meta = extract_video_meta(message) # type: ignore[arg-type]
163         if meta is None:
164             continue
165
166         channel_id = _peer_id(getattr(message, "peer_id", None) or 0)

```

56. assistant (2026-07-06T23:56:15.204Z)

The `try` at 147 wraps the whole `async for` message loop; the matching `except` is `FloodWaitError` at 327 (only). So a `TypeError` from `\_record\_policy\_decision` inside the loop is NOT caught by that block ? it propagates up to the outer `except Exception` at 464 (`backfill\_crashed`), aborting the entire backfill for that user. Same crash class, contained only at the top level. Reinforces the serialization finding for the backfill path too.

I have enough to report. Let me finalize. Summary of findings on the persistence write-path / serialization / wiring dimension:

****Confirmed reproductions:****

- Non-serializable stage metadata ? `json.dumps` TypeError propagates uncaught out of `db.record\_policy\_decision` ? out of `\_record\_policy\_decision` (no try/except in either `on\_new\_message` or the backfill message loop) ? crashes ingestion / aborts backfill. Latent (current F1-F5 stages all emit JSON-safe metadata; the exposed path is `\*\*exc.metadata` in `DeepInspectionStage` at policy.py:763 fed by a future F5 ContentFetcher/budget). Rubric places "serialization crash" at medium.
- FK is enforced (`PRAGMA foreign\_keys = ON`); wiring records only after `insert\_video`, so content_item_id is valid; dedup-race path returns before recording (no orphan). Rejected items persist `passed=False` with no delivery_attempt. All correct.

****Notes (low):**** metadata-rejected (Tier-1) items get no decision row (F7 /why gap); unbounded growth/no TTL; guard early-return and non-serializable paths untested.

Let me report.

57. user (2026-07-06T23:56:40.190Z)

Structured output provided successfully